Document-Oriented Databases

- Document-Oriented Databases Management Systems (DODBMS) have evolved from Key-Value Databases
- DODBMS are used for storing, retrieving and managing semi-structured data
- DODBMS propose a schema-less flexible data representation for data providing more flexibility for data modeling

Document-Oriented Databases

- DODBMS use documents for storing data
- The term document may usually refer a Word or PDF documents, but in this case refers to JSON or XML files
- The flexibility provided by XML and JSON make it easier to modify documents than it is for tables in a RDBMS
- Documents are stored in collections

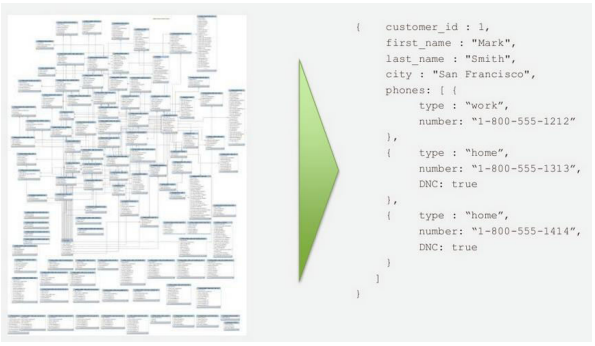
Document-Oriented Databases

- In DODBMS information is stored using fields (labels, tags, descriptions) to describe the data and values for that description
- New information (fields) can be added directly to a record without the need to modify the entire schema as it is for RDBMS

RDBMS vs DODBMS Comparison

RDBMS	DODBMS
Database (Schema)	Database
Table	Collection
Attribute	Field (label, tag, description)
Attribute values	Field (label, tag, description) value

RDBMS vs DODBMS Comparison



Benefits of a DODBMS

- Flexible data modeling
- Fast write performance
- Fast query performance

Benefits of a DODBMS Flexible data modeling

- Data from the web, mobile, social and IoT applications change the nature of the application's data model
- In a RDBMS, this changes impose the modification of the schema by altering tables and adding or removing new columns
- The Flexibility of DODBMS eliminate the need to force-fit the data into predefined attributes and tables

Benefits of a DODBMS Fast write performance

- Some DODBMS prioritize write availability over strict data consistency
- This ensures that writes will always be executed even if there is a hardware or network failure
- In case of failure, the replication and eventual consistency mechanisms ensures that the environment will function

Benefits of a DODBMS
Fast query performance

- Most DODBMS
  - Provide powerful query engines for CRUD operations
  - Use indexes and secondary indexes to improve data retrieval
  - Provide aggregation frameworks for Data Analysis and Business Intelligence
    - Aggregation is either native or done through MapReduce

MongoDB – Overview

- MongoDB uses a flexible, dynamic schema to store data
- A MongoDB instance is started using mongod
- A record is stored in a document
- Multiple documents are stored in a collection

MongoDB
Data representation

- MongoDB stores the data in BSON documents
- BSON is a binary-encoded serialization of JSON-like documents
  - Note: Serialization is the process of translating data structures or object state into a format that can be stored (e.g., in a file, memory buffer, etc.) or transmitted (e.g., across a network connection link, etc.) and reconstructed later (possibly in a different computer environment)
- A BSON document is limited in size to 16MB
- Solution to the size limitation MongoDB GridFS

```
{"hello": "world"} →
\x16\x00\x00\x00          // total document size
\x02                      // 0x02 = type String
hello\x00                 // field name
\x06\x00\x00\x00world\x00 // field value
\x00                      // 0x00 = type EOO ('end of object')
```


MongoDB
Key Features

- Support of multiple storage engines
- High availability through replication
- Automatic horizontal scaling though sharding
- High performance
- Rich query language

MongoDB – Overview

- MongoDB (from humongous) is focused on combining the critical capabilities of RDBMS with the innovations of NoSQL DBMS
- MongoDB is an open-source Document-Oriented Database Management System (DODBMS)
- MongoDB is written in C++, C and JavaScript
- Run on different OS: Linux (most distributions), Windows, MacOS
- Can be easily deployed in the Cloud as a DaaS (MongoDB Atlas, AWS, and MongoDB containers)

MongoDB – Overview

- Documents in a collection do not necessarily have the same structure:
  - The number of attributes can differ from one record to another
  - The fields' data type can differ from one record to another
- Although the database allows documents with a different number of attributes and for the same fields different data types, in the same collection records have almost the same structure
- In practice, documents usually model objects from a high-level programming language

MongoDB
Data representation

- The main features of BSON are:
  - Lightweight – for fast retrieval of information over a computer network it is necessary to keep the overhead needed to store data to a minimum
  - Traversable – it is import to store data in a format that is easily parsed
  - Efficient – using C language data types, data is easily encoded and decoded

MongoDB
Key Features

- MongoDB supports multiple storage engines:
  - MMAPv1 Storage Engine (deprecated starting MongoDB v4.0, removed in MongoDB v4.2)
  - WiredTiger Storage Engine
  - In-Memory Storage

MongoDB Key Features

- High performance data persistence
  - Support for embedded data models reduces I/O activity on database systems:
    - Supports nested documents
    - References can also be used
  - Indexes support faster queries
  - Indexes can include keys from embedded documents and arrays
  - Indexes can be added on text and geospatial data
  - MongoDB support compound indexes

MongoDB Key Features

- Note on Indexes:
  - Index prefix is the label that defines the index, e.g.:
    {idx_1: 1}
  - Compound indexes use multiple fields in their structure, e.g.:
    {idx_1: 1, idx_2: 1, idx_3: 1}
  - Queries that use efficiently a compound index use the labels in order, e.g.
    - idx_1,
    - idx_1 and idx_2
    - idx_1 and idx_2 and idx_3

MongoDB Key Features

- Note on Indexes:
  - Also works for our example:
    - idx_1 and idx_3
  - Not as efficient as creating a compound index only on idx_1 and idx_3
  - Queries that don't use the first label(s) (i.e., idx_1), don't use the compound index
    - idx_2
    - idx_3
    - idx_2 and idx_3
    - idx_3 and idx_2

MongoDB Key Features

- Rich query language
  - Supports all the CRUD Operations
  - Supports data aggregation through MapReduce and Aggregation Pipeline
  - Supports Text Search and regular expressions
  - Supports Geospatial queries

MongoDB Key Features

- High availability
  - Supports replication, called Replica Set, which provide:
    - Automatic failover
    - Data redundancy
    - Increased data availability
  - A Replica Set is a group of MongoDB servers that maintain the same data set

MongoDB Key Features

- Horizontal Scalability
  - MongoDB provides horizontal scalability as part of its core functionality
  - Scalability is achieved through Sharding
  - Sharding distributes data across a cluster of machines
  - Form MongoDB 3.4 the support of creating zones of data on the shard key was added
  - Using zones in a balanced cluster, MongoDB directs reads and writes covered by a zone only to those shards inside the zone

MongoDB – MMAPv1

- MMAPv1 Storage Engine
  - The original storage engine
  - It excels at workloads with high volume of inserts, reads and in-place updates
  - Deprecated in v4.0, removed in v4.2
  - Features:
    - Journal
    - Record storage characteristics
    - Record allocation strategies
    - Memory use

MongoDB – MMAPv1 Journal

- To ensure data persistence, all modifications are registered to an on-disk Journal
- Journal writes happen more frequently than data files writes
- In the default configuration of the MMAPv1 there are Journal writes roughly every 100ms and data file writes every 60s
- To change the write intervals, change the values for the following parameters:
  - Journal file : storage.journal.commitIntervalMs
  - Data file: storage.syncPeriodSecs

MongoDB – MMAPv1
Journal

- The interval values represent the maximum amount of time between
  - The completion of a write operation and
  - When MongoDB writes to the data files or to the journal files
- In many cases MongoDB and the operating system flush data to disk more frequently (the above values represents a theoretical maximum)
- The journal allows MongoDB to successfully recover data from data files after a mongod instance exits without flushing all changes

MongoDB – MMAPv1
Record allocation strategies

- MongoDB supports multiple record allocation strategies that determine how mongod adds padding to a document when creating a record
- Because documents in MongoDB may grow after insertion and all records are contiguous on disk, the padding can reduce the need to relocate documents on disk following updates
- Relocations
  - Are less efficient than in-place updates
  - Can lead to storage fragmentation
- As a result, all padding strategies trade additional space for
  - Increased efficiency
  - Decreased fragmentation

MongoDB – MMAPv1
Record allocation strategies

- There are two allocations strategies
  - Power of 2 allocations
  - No padding allocation strategy (or exact fit allocations)
- The two different allocation strategies support different kinds of workloads:
  - The power of 2 allocations are more efficient for insert/update/delete workloads
  - Exact fit allocations is ideal for collections without update and delete workloads

MongoDB – MMAPv1
Record storage characteristics

- MongoDB uses Power of 2 Sized Allocations by default
- Every document in MongoDB is stored in a record which contains:
  - The document itself
  - Extra space, or padding
- Padding
  - Allows the document to grow as the result of updates
  - Minimizing the likelihood of reallocations

MongoDB – MMAPv1
Power of 2 Sized Allocations

- Appeared with MongoDB 3.0
- Uses the power of 2 sizes allocation as the default record allocation strategy for MMAPv1
- With this strategy, each record has a size in bytes that is a power of 2 (e.g., 32, 64, 128, 256, 512 … 2 MB)
- For documents larger than 2 MB, the allocation is rounded up to the nearest multiple of 2 MB

MongoDB – MMAPv1
Power of 2 Sized Allocations

- Properties
  - Can efficiently reuse freed records to reduce fragmentation:
    - Quantizing record allocation sizes into a fixed set of sizes increases the probability that an insert will fit into the free space created by an earlier document deletion or relocation.
  - Can reduce moves:
    - The added padding space gives a document room to grow without requiring a move
    - In addition to saving the cost of moving, this results in less updates to indexes
    - Although the power of 2 sizes strategy can minimize moves, it does not eliminate them entirely

MongoDB – MMAPv1
No Padding Allocation Strategy

- Is used for collections whose workloads do not change the document sizes
- Workloads consist of
  - Insert-only operations
  - Update operations that do not increase document size
- Prior MongoDB v3.0.0, the allocation strategy included a dynamically calculated padding as a factor of the document size

MongoDB – MMAPv1
Memory use

- MongoDB automatically uses all free memory on the machine as its cache
- System resource monitors show that MongoDB uses a lot of memory (its usage is dynamic)
- If another process suddenly needs half the server's RAM, MongoDB will yield cached memory to the other process

MongoDB – MMAPv1 Memory use

- The operating system's virtual memory subsystem manages MongoDB's memory
- This means that MongoDB will use as much free memory as it can, swapping to disk as needed
- Deployments with enough memory to fit the application's working data set in RAM will achieve the best performance

MongoDB – WiredTiger

- The default storage for the latest versions
- Introduced from MongoDB v3.0, main Storage Engine from v4.2
- Features:
  - Document level concurrency
  - Snapshots and checkpoints
  - Journal
  - Compression
  - Memory use

MongoDB – WiredTiger Document level concurrency

- WiredTiger uses document-level concurrency control for write operations
- Multiple clients can modify different documents of a collection at the same time
- WiredTiger uses optimistic concurrency control (optimistic locking) for most read and write operations

MongoDB – WiredTiger Document level concurrency

- MongoDB uses reader-writer locks that allow concurrent readers shared access to a resource, such as a database or collection
- Locking modes create a lock on the resource

MongoDB – WiredTiger Document level concurrency

- Shared Locking mode for reads
  - Allow multiple transactions to read the same data, but prevent any transaction from modifying it

MongoDB – WiredTiger Document level concurrency

- Exclusive Locking mode for write
  - Protect updates to file resources, both recoverable and non-recoverable
  - Can be owned by only one transaction at a time
  - Any transaction that requires an exclusive lock must wait if another task currently owns an exclusive lock or a shared lock against the requested resource.

MongoDB – WiredTiger Document level concurrency

- When locking at a certain granularity, all higher levels are locked using an intent lock
- Intent locks
  - Are used by a transaction to inform another transaction about its intention to acquire a lock
  - Ensure that data modification are executed properly by preventing another transaction to acquire a lock on the next in hierarchy object
- WiredTiger uses intent locks only at the Global, Database, and Collection levels
- When the storage engine detects conflicts between two operations, one will incur a write conflict causing MongoDB to transparently retry that operation

MongoDB – WiredTiger Document level concurrency

- Intent Shared Locking mode to indicate an intent to read using a finer granularity lock
  - Indicates that the transaction has the intention to read some lower hierarchy resources by acquiring Shared Locks individually on those resources lower in the hierarchy
  - Example: When a transaction locks a document using a Shared Lock mode (to read the date), the database server places an Intent Shared lock on the collection to provide an instant check that no other transaction holds an Exclusive Lock on the collection

MongoDB – WiredTiger
Document level concurrency

- Intent Exclusive Locking mode to indicate an intent write using a finer granularity lock
  - Indicates the intention of a transaction to modify some (but not all) resources lower in the hierarchy by placing Exclusive locks on those individual resources
  - Example:
    - When a transaction tries to modify a record using an Exclusive Lock, an Intent Exclusive Lock is put in place on the collection
    - The lock cannot be put in place until all other locks have been released on this record (or collection)
    - While the exclusive lock is in place, no other intent locks can be applied on the record (or collection)

MongoDB – WiredTiger
Document level concurrency

- A single database can simultaneously be locked using an Intent Shared Lock and Intent Exclusive Lock
- An Exclusive Lock cannot coexist with any other locking modes
- A Shared Lock can only coexist with Intent Shared Locks
- For example, when locking a collection for writing (using an Exclusive Lock), both the corresponding database lock and the global lock must be locked using Intent Exclusive Locking

MongoDB – WiredTiger
Document level concurrency

- Global "instance-wide" locks are required in general for some global operations
  - E.g., short-lived operations involving multiple databases
- Exclusive database locks are required for other operations
  - E.g., dropping a collection

MongoDB – WiredTiger
Snapshots and checkpoints

- WiredTiger uses MultiVersion Concurrency Control (MVCC)
  - MVCC is a database optimization technique that creates duplicate copies of records so that data can be safely read and updated at the same time
  - Also available for other databases, e.g., Couchbase
- WiredTiger provides a point-in-time snapshot of the data to the transaction at the start of an operation
- A snapshot presents a consistent view of the in-memory data

MongoDB – WiredTiger
Snapshots and checkpoints

- When writing to disk, WiredTiger writes all the data in a snapshot to disk in a consistent way across all data files
- The now-durable data act as a checkpoint in the data files
- The checkpoint ensures that the data files are consistent up to and including the last checkpoint (i.e., checkpoints can act as recovery points)
- MongoDB configures WiredTiger to create checkpoints (i.e., write the snapshot data to disk) at intervals of 60 seconds or 2 gigabytes of journal data

MongoDB – WiredTiger
Snapshots and checkpoints

- During the write of a new checkpoint, the previous checkpoint is still valid
- As such, even if MongoDB terminates or encounters an error while writing a new checkpoint, upon restart, MongoDB can recover from the last valid checkpoint
- The new checkpoint becomes accessible and permanent when WiredTiger's metadata table is atomically updated to reference the new checkpoint
- Once the new checkpoint is accessible, WiredTiger frees pages from the old checkpoints

MongoDB – WiredTiger
Snapshots and checkpoints

- MongoDB can recover from the last checkpoint when using WiredTiger, even without journaling
- However, to recover changes made after the last checkpoint, MongoDB should be run with journaling
- From v4.2, the following options are not available anymore for replica set members :
  - --nojournal parameter for the mongod demon
  - storage.journal.enabled: false in the configuration file

MongoDB – WiredTiger
Snapshot History Retention

- Starting in MongoDB 5.0, you can use the minSnapshotHistoryWindowInSeconds parameter to specify how long WiredTiger keeps the snapshot history.
- Increasing the value of minSnapshotHistoryWindowInSeconds increases disk usage because the server must maintain the history of older modified values within the specified time window.
- The amount of disk space used depends on your workload, with higher volume workloads requiring more disk space.
- MongoDB maintains the snapshot history in the WiredTigerHS.wt file, located in your specified dbPath.

MongoDB – WiredTiger Journal

- WiredTiger uses checkpoints to
  - Provide a consistent view of data on disk
  - Allow MongoDB to recover from the last checkpoint
- However, if MongoDB exits unexpectedly in between checkpoints, journaling is required to recover information that occurred after the last checkpoint

MongoDB – WiredTiger Journal

- The recovery process with journaling:
  - Looks in the data files to find the identifier of the last checkpoint
  - Searches in the journal files for the record that matches the identifier of the last checkpoint
  - Apply the operations in the journal files since the last checkpoint

MongoDB – WiredTiger Journaling Process

- WiredTiger creates one journal record for each client who initiated write operations
  - The journal record includes any internal write operations caused by the initial write
- WiredTiger creates a single journal record that includes both the update operation and its associated index modifications
- MongoDB configures WiredTiger to use in-memory buffering for storing the journal records
- Threads coordinate to allocate and copy into their portion of the buffer
- All journal records up to 128 kB are buffered

MongoDB – WiredTiger Journaling Process

- WiredTiger syncs the buffered journal records to disk according to the following intervals or conditions:
  - Every 50 milliseconds from MongoDB v3.2
  - MongoDB sets checkpoints to occur in WiredTiger on user data at an interval of 60 seconds or when 2 GB of journal data has been written, whichever occurs first.
  - WiredTiger forces a sync of the WiredTiger journal files if the write operation includes a write concern that requests the acknowledgement from MongoDB that the write operation has been written to the journal

MongoDB – WiredTiger Journaling Process

- WiredTiger syncs the buffered journal records to disk according to the following intervals or conditions:
  - When WiredTiger creates a new journal file, WiredTiger syncs the previous journal file
  - WiredTiger creates a new journal file approximately every 100 MB of data
    - This happens because MongoDB uses a journal file size limit of 100 MB

MongoDB – WiredTiger Journal Files

- For the journal files, MongoDB creates a subdirectory named journal under the database directory (in this directory the data is stored)
- WiredTiger journal files have names with the following format WiredTigerLog.<sequence> where <sequence> is a zero-padded number starting from 0000000001
- Journal files contain a record per each write operation
- Each record has a unique identifier
- MongoDB configures WiredTiger to use snappy compression for the journaling data
  - Snappy is a fast data compression and decompression library written in C++ by Google based on ideas from LZ77 and open-sourced in 2011.

MongoDB – WiredTiger Journal Files

- Minimum log record size for WiredTiger is 128 bytes
- If a log record is 128 bytes or smaller, WiredTiger does not compress that record
- WiredTiger journal files have a maximum size limit of approximately 100 MB
- WiredTiger creates a new journal file once the file exceeds that limit
- WiredTiger automatically removes old journal files to maintain only the files needed to recover from last checkpoint
- WiredTiger will pre-allocate journal files.

MongoDB – WiredTiger Compression

- WiredTiger supports compression for all collections and indexes
- Compression minimizes storage use at the expense of additional CPU
- By default, WiredTiger uses
  - Block compression with the snappy compression library for all collections
  - Prefix compression for all indexes
- For collections, block compression with zlib is also available

MongoDB – WiredTiger Compression

- To specify an alternate compression algorithm or no compression, use the following setting:

storage.wiredTiger.collectionConfig.blockCompressor

- For indexes, to disable prefix compression, use the following setting:

storage.wiredTiger.indexConfig.prefixCompression

- Compression settings are also configurable on a per-collection and per-index basis during collection and index creation

MongoDB – WiredTiger Compression

- The default compression settings balance storage efficiency and processing requirements for most workloads
- As already mentioned, the WiredTiger journal is also compressed by default

MongoDB – WiredTiger Memory use

- MongoDB utilizes both the WiredTiger internal cache and the filesystem cache
- Starting with MongoDB v3.4, the WiredTiger internal cache, by default, will use the larger of either:
  - 50% of RAM minus 1 GB, or
  - 256 MB

MongoDB – WiredTiger Memory use

- Different representations are used for data in the WiredTiger internal cache versus the on-disk format
  1. Data in the filesystem cache is the same as the on-disk format
     - This method benefits of any compression for data files
     - The filesystem cache is used by the operating system to reduce disk I/O
  2. Indexes loaded in the WiredTiger internal cache have a different data representation to the on-disk format
     - This method can still take advantage of index prefix compression to reduce RAM usage
     - Index prefix compression de-duplicates common prefixes from indexed fields

MongoDB – WiredTiger Memory use

3. Collection data in the WiredTiger internal cache is uncompressed and uses a different representation from the on-disk format
   - Block compression can provide significant on-disk storage savings
   - The data must be uncompressed to be manipulated by the server

In-Memory Storage Engine

- First release in MongoDB Enterprise version 3.2.6
- The in-memory storage engine is part of general availability (GA) in the 64-bit builds.
- Other than some metadata and diagnostic data, the in-memory storage engine does not maintain any on-disk data, including configuration data, indexes, user credentials, etc.
- By avoiding disk I/O, the in-memory storage engine allows for more predictable latency of database operations

In-Memory Storage Engine

- Concurrency
  - The in-memory storage engine uses document-level concurrency control for write operations
  - Multiple clients can modify different documents of a collection at the same time
- Memory Use
  - In-memory storage engine requires that all its data (including indexes, oplog if mongod instance is part of a replica set, etc.) must fit into the specified amount of memory set when starting the instance
  - By default, the in-memory storage engine uses 50% of physical RAM minus 1 GB
  - If a write operation would cause the data to exceed the specified memory size, MongoDB returns an error

In-Memory Storage Engine

- Durability
  - The in-memory storage engine is non-persistent and does not write data to a persistent storage
  - Non-persisted data includes application data and system data, such as users, permissions, indexes, replica set configuration, sharded cluster configuration, etc.
  - The concept of journal or waiting for data to become durable does not apply to the in-memory storage engine
  - The recovery of in-memory data is impossible
- Transactions
  - Starting in MongoDB 4.2, transactions are supported on replica sets and sharded clusters where:
    - The primary uses the WiredTiger storage engine
    - The secondary members use either the WiredTiger storage engine or the in-memory storage engines

In-Memory Storage Engine Deployment Architectures

- Standalone
- MongoDB instances that use in-memory storage engine can run as part of
  - A replica set
  - A sharded cluster

In-Memory Storage Engine Deployment Architectures – Replica Set

- Create multiple in-memory instances that can become primary
- Create at least 1 hidden WireTiger instance as secondary to avoid losing data
- Set the priority of the hidden instance(s) to 0 so it will never become a primary
- Only the instances running with the in-memory storage engine can become the primary
- Clients can connect only to the in-memory storage engine instances
- Even if all instances running in-memory storage engine crash and restart, they can sync from the member running WiredTiger
- The instance running with WiredTiger persists the data to disk, including the user data, indexes, and replication configuration information.

In-Memory Storage Engine Deployment Architectures – Sharded Cluster

- Create at least 1 replica set that you will use as a shard
  - Create multiple in-memory instances
  - Create at least 1 hidden WireTiger instance
  - Set the priority of the hidden instance(s) to 0 so it will never become a primary
- Create normal replica sets without in-memory instances
- The shard(s) with in-memory instances are called the in-memory shards
- The other shards (that do not use in-memory instances) become persistent shards
- For each sharded collection that should reside on the in-memory shard(s), assign to the entire chunk range
- For each sharded collection that should reside across the persisted shards, assign to the entire chunk range

MongoDB – GridFS

- GridFS is a specification for storing and retrieving files that exceed the BSON-document size limit of 16 MB
- Instead of storing a file in a single document, GridFS divides the file into parts, or chunks
- Each chunk is stored as a separate document (in two separate collections)
- GridFS is useful for:
  - Storing files that exceed 16 MB
  - Storing any files for which you want access without having to load the entire file into memory.

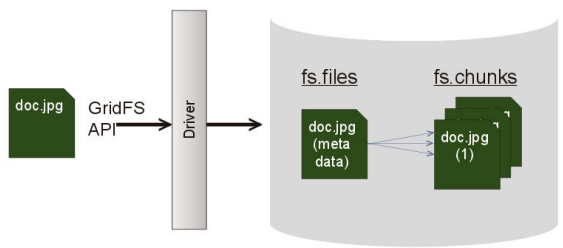
MongoDB – GridFS

- GridFS uses a chunk size of 255 kB by default
  - GridFS divides a file into chunks of 255 kB with the exception of the last chunk
  - The last chunk is only as large as necessary
  - Similarly, files that are no larger than the chunk size
    - Only have a final chunk
    - Use only as much space as needed plus some additional metadata

MongoDB – GridFS

- GridFS uses two collections to store files.
  - One collection stores the file chunks
  - The other stores file metadata
- The driver will reassemble the chunks as needed when querying a file stored on GridFS
- Range queries can be performed on files stored on GridFS
- Information from arbitrary sections of files can also be accessed:
  - E.g., If a video or audio file is stored then you can skip through them

MongoDB – GridFS



MongoDB – GridFS When to use MongoDB GridFS

- In some situation is more efficient to store large files in a MongoDB database than on a System-Level Filesystem:
  - If the filesystem limits the number of files in a directory
  - To access information from portions of large files without having to load whole files into memory
  - To keep files and their metadata automatically synced and deployed across several systems and facilities

MongoDB – GridFS
When NOT to use MongoDB GridFS

- If the content of the entire file needs to be updated atomically
- As an alternative:
  - Store multiple versions of each file and specify the current version of the file in the metadata
  - Update the metadata field that indicates the "latest" status in an atomic update after uploading the new version of the file
  - The previous versions of the file can be removed if needed

MongoDB – GridFS

- If your files are all smaller than the 16 MB BSON Document Size limit:
  - Consider storing each file in a single document instead of using GridFS
  - The BinData data type can be used to store the binary data
  - The driver documentation presents all the details on how to use BinData
- MongoDB Driver:
  - An application communicates with MongoDB by way of a client library called a driver
  - Handles all interaction with the database in a language appropriate to the application (e.g., Python, C, C++, Java, etc.)

MongoDB – Replication

- mongod is the instance of a MongoDB server
- A replica set in MongoDB is a group of mongod processes that maintain the same data set.
- Replica sets provide
  - Redundancy
  - High availability
- Replica sets are the basis for all production deployments

MongoDB – Replication

- Replication provides
  - Redundancy
    - With multiple copies of data on different database servers, replication provides a level of fault tolerance against the loss of a single database server
  - High availability:
    - In some cases, replication can provide increased read capacity as clients can send read operations to different servers

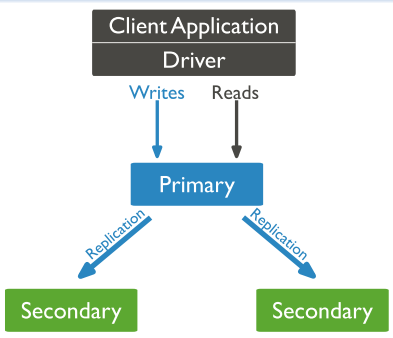
MongoDB – Replication

- Maintaining copies of data in different data centers can increase
  - Data locality
  - Availability for distributed applications
- Additional copies can be maintained for dedicated purposes, such as:
  - Disaster recovery
  - Reporting
  - Backup

MongoDB – Replication

- A replica set is a group of mongod instances that maintain the same data set
- A replica set contains
  - Several data bearing nodes
  - Optionally one arbiter node
- The data bearing nodes
  - One and only one member is deemed the primary node
  - The other nodes are deemed secondary nodes
- The primary node
  - Receives all write operations
  - Records all changes to its data sets in its operation log

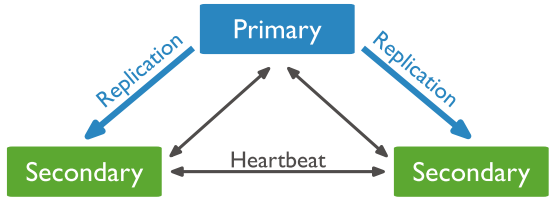
MongoDB – Replication



MongoDB – Replication

- The secondaries
  - Replicate the primary's oplog (operations log)
    - oplog: A special capped collection that keeps a rolling record of all operations that modify the data stored in the databases
    - capped collection: A fixed-sized collection that automatically overwrites its oldest entries when it reaches its maximum size
  - Apply the operations to their data sets such that the secondaries' data sets reflect the primary's data set
- If the primary is unavailable, an eligible secondary will hold an election to elect itself the new primary

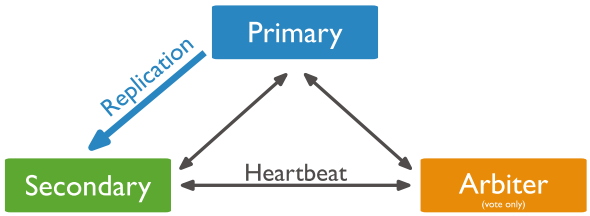
MongoDB – Replication



MongoDB – Replication

- To maintain the quorum of votes in a replica set an Arbiter can be added
- Arbiters
  - Do not maintain a data set
  - Respond to heartbeat and election requests by other replica set members
  - Can be a good way to provide replica set quorum functionality with a cheaper resource cost than a fully functional replica set member with a data set

MongoDB – Replication



MongoDB – Replication



MongoDB – Replication

- If your replica set has an even number of members, add an arbiter to obtain most votes in an election for primary
- Arbiters do not require dedicated hardware
- An arbiter will always be an arbiter whereas
  - A primary may step down and become a secondary during an election
  - A secondary may become the primary during an election

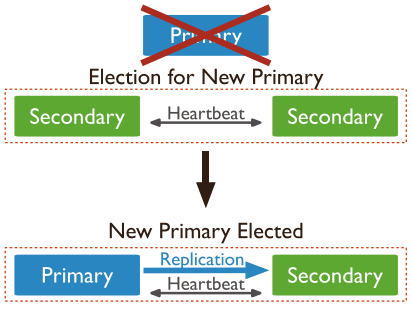
MongoDB – Replication

- Asynchronous Replication
  - Secondaries are updated by applying operations from the primary asynchronously
  - By applying operations after the primary, sets can continue to function despite the failure of one or more members

MongoDB – Replication

- Automatic Failover
  - If the primary node does not respond to a heartbeat for more than 10 seconds an eligible secondary holds an election to elect itself the new primary
  - The first secondary to hold an election and receive most of the members' votes becomes primary

MongoDB – Replication



MongoDB – Replication

- Replication Read Operations
  - By default, clients read from the primary
  - However, clients can specify to read from Secondaries
  - Because of asynchronous replication
    - There is no guaranty that secondaries have the latest information
    - The latest information is only on the primary node
  - Clients can see the results of writes before the write becomes durable

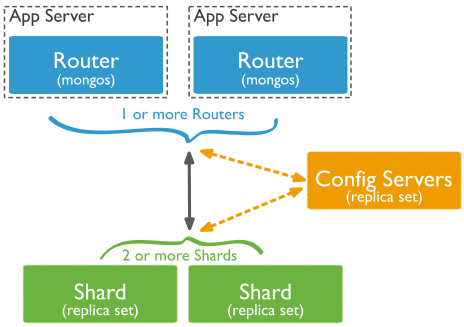
MongoDB – Sharding

- Sharding is a method for distributing data across multiple machines
- MongoDB uses sharding to support deployments with very large data sets and high throughput operations
- Database systems with large data sets or high throughput applications can challenge the capacity of a single server
- High query rates can exhaust the CPU capacity of the server
- Working set sizes larger than the system's RAM stress the I/O capacity of disk drives
- MongoDB supports horizontal scaling through sharding
- MongoDB shards data at the collection level, distributing the collection data across the shards in the cluster

MongoDB – Sharding

- A MongoDB sharded cluster consists of the following components:
  - shard:
    - Each shard contains a subset of the sharded data
    - Each shard must be deployed as a replica set
  - mongos:
    - The mongos acts as a query router
    - Provides an interface between client applications and the sharded cluster
  - config servers:
    - Config servers store metadata and configuration settings for the cluster
    - As of MongoDB 3.4, config servers must be deployed as a replica set (Configuration Servers Replica Set - CSRS)

MongoDB – Sharding



MongoDB – Sharding

- Shard Keys
  - To distribute the documents in a collection, MongoDB partitions the collection using the shard key
  - The shard key consists of an immutable field or fields that exist in every document in the target collection
  - The shard key is chosen when sharding the collection
  - The choice of shard key cannot be changed after sharding
  - The shard key can be a composite key
  - A sharded collection can have only one shard key

MongoDB – Sharding

- Shard Keys
  - To shard a non-empty collection:
    - The collection must have an index that starts with the shard key
  - To shard empty collections:
    - MongoDB creates the index if the collection does not already have an appropriate index for the specified shard key

MongoDB – Sharding

- Shard Keys
  - For a sharded cluster, the choice of shard key affects the:
    - Performance
    - Efficiency
    - Scalability
  - A cluster with the best possible hardware and infrastructure can be bottlenecked by the choice of shard key
  - The choice of shard key and its backing index can also affect the sharding strategy that your cluster can use

MongoDB – Sharding

- Chunks
  - MongoDB partitions sharded data into chunks
  - Each chunk has an inclusive lower and exclusive upper range based on the shard key
  - MongoDB migrates chunks across the shards in the sharded cluster using the sharded cluster balancer
  - The balancer attempts to achieve an even balance of chunks across all shards in the cluster

MongoDB
Advantages of sharding

- Read/Write:
  - MongoDB distributes the read and write workload across the shards in the sharded cluster
  - Each shard processes a subset of cluster operations
- Storage Capacity:
  - Sharding distributes data across the shards in the cluster
  - Each shard contains a subset of the total cluster data
- High Availability:
  - A sharded cluster can continue to perform partial read / write operations even if one or more shards are unavailable

Starting a MongoDB instance

- When mongo is started the default host is localhost (127.0.0.1) and the default port is 27017
- If you want to change the host, when you start mongod use --bind_ip and give an IP

```
$ mongod --dbpath mongo_data --bind_ip IP
```

- If there is a binding between the host name and an IP (in /etc/hosts) you can use the hostname instead of the IP to connect to MongoDB
- If you want to change the default port, when you start mongod use --port and give a new port

```
$ mongod --dbpath mongo_data --port 27110
```


Connecting to MongoDB

- To connect to MongoDB use mongosh

```
$ mongosh [--host {IP|hostname}] [--port {port_no}] [DB_name]
```

```
$ mongosh [{IP|hostname}:{port_no}/][DB_name]
```

- If no database name is specified, then mongo will connect directly to the admin database

Replication in MongoDB

- Start multiple instances using mongod with the --replSet ReplicaName parameter
- Local replication example

1. Create a data directory for each node (make sure to have the privileges):

```
$ mkdir -p node1_data node2 _data node3_data
```

2. Start the instances:

```
$ nohup mongod --replSet rs1 --dbpath node1_data --port 27021 > log_n1 2>&1 &
$ nohup mongod --replSet rs1 --dbpath node2_data --port 27022 > log_n2 2>&1 &
$ nohup mongod --replSet rs1 --dbpath node3_data --port 27023 > log_n3 2>&1 &
```


Replication in MongoDB

- Initialize the replication
- Connect to one of the nodes:

```
$ mongosh localhost:27021
```

- Initialize the replica set

```
> rs.initiate( {
   _id: "rs1",
   members: [ { _id : 0, host : "localhost:27021" }, { _id : 1, host
:  "localhost:27022" }, { _id : 2, host : "localhost:27023" }  ] } )
```

- Verify the status of the replica set

```
> rs.status()
```


Replication in MongoDB

- To read from a secondary:
  - Connect to the secondary node
  - Issue the following command

```
> rs.secondaryOk() // depricated
> db.getMongo().setReadPref("primaryPreferred")
```

  - Disable reading from secondary

```
> db.getMongo().setReadPref("primary")
```


Sharding in MongoDB

- Create the directories for the config and shard nodes

```
$ mkdir -p conf1_data conf2_data conf3_data
$ mkdir -p rs1node1_data rs1node2_data rs1node3_data
$ mkdir -p rs2node1_data rs2node2_data rs2node3_data
$ mkdir -p rs3node1_data rs3node2_data rs3node3_data
```

- Create the configuration servers

```
$ nohup bin/mongod --configsvr --replSet ConfRepl --dbpath conf1_data --port
27121 > log_conf1 2>&1 &
$ nohup bin/mongod --configsvr --replSet ConfRepl --dbpath conf2_data --port
27122 > log_conf2 2>&1 &
$ nohup bin/mongod --configsvr --replSet ConfRepl --dbpath conf3_data --port
27123 > log_conf3 2>&1 &
```


Sharding in MongoDB

- Initialize replication for the config servers

```
$ mongosh localhost:27121
> rs.initiate(  {
   _id: "ConfRepl",
   configsvr: true,
   members: [
                    { _id : 0, host : "localhost:27121" },
                    { _id : 1, host : "localhost:27122" } ,
                    { _id : 2, host : "localhost:27123" }
          ]
} )
```

Sharding in MongoDB

- Start the sharding replica set nodes

```
$ nohup bin/mongod --shardsvr --replSet RS1 --dbpath rs1node1_data --port 27011 > log_rs1node1 2>&1 &
$ nohup bin/mongod --shardsvr --replSet RS1 --dbpath rs1node2_data --port 27012 > log_rs1node2 2>&1 &
$ nohup bin/mongod --shardsvr --replSet RS1 --dbpath rs1node3_data --port 27013 > log_rs1node3 2>&1 &
```

- And initialize replication

```
$ bin/mongosh localhost:27011
> rs.initiate({
   _id: "RS1",
   members: [
      { _id : 0, host : "localhost:27011" },
      { _id : 1, host : "localhost:27012" },
      { _id : 2, host : "localhost:27013" }
   ]
})
```

Note: Do this 2 more times for the other 2 shards – change the configuration accordingly for each shard

Sharding in MongoDB

- Start a mongos instance

```
$ nohup bin/mongos --configdb ConfRepl/localhost:27121,localhost:27122,localhost:27123 --port 27130 > /home/log_mongos 2>&1 &
```

- Connect to the mongos instance and add the shards

```
$ bin/mongosh localhost:27130

> sh.addShard("RS1/localhost:27011")
> sh.addShard("RS1/localhost:27012")
> sh.addShard("RS1/localhost:27013")
> sh.addShard("RS2/localhost:27021")
> sh.addShard("RS2/localhost:27022")
> sh.addShard("RS2/localhost:27023")
> sh.addShard("RS3/localhost:27031")
> sh.addShard("RS3/localhost:27032")
> sh.addShard("RS3/localhost:27033")
```


Sharding in MongoDB

- Verify the sharding status

```
> sh.status()
```

- Create a collection

```
> use BD2
> var vec = [ {x: 0 }]
> vec.length
> for (var i=1; i<100000000; i++){ vec.push({x: i})}
> vec.length
> db.documents.insertMany(vec);
> db.documents.findOne()
```


Sharding in MongoDB

- Enable sharding using on a database **sh.enableSharding("db_name")**

```
sh.enableSharding("BD2")
```

- Shard the collection using **sh.shardCollection("db_name.collection_name", {key: 1})**

```
sh.shardCollection("BD2.documents", {_id: 1})
```

- Verify the sharding status

```
sh.status({verbose: true})
```


Sharding in MongoDB Conclusion

- Start the config servers
- Initialize replication for the config servers
- Create shard replicas
- Initialize replication for each shard replica set
- Start mongos
- Add the replica shards

CRUD Operations in MongoDB

- Insert operation:
  - db.collection_name.insertOne(doc)
  - db.collection_name.insert(doc, {ordered: <boolean>}))
  - db.collection_name.insert([docs])
  - db.collection_name.insertMany([docs], {ordered: <boolean>})

CRUD Operations in MongoDB

- Insert Examples

```
> doc = { name: {firstname: "Ciprian", lastname: "Truica"}, age: 32, courses: [ {name: "BD1", year: 3}, {name: "BD2", year: 4}, {name: "DM", year: 5}], offices: ["EF208", "PR305"]}

> db.teachers.insert(doc)

> doc = { name: {firstname: "Alexandru", lastname: "Boicea"}, age: 59, courses: [{ name: "BD1", year: 3}, { name: "BD2", year: 4}, { name: "SCAD", year: 4}, { name: "ICASE", year: 5}], offices: ["EF309"]}

> db.teachers.insertOne(doc)
```


CRUD Operations in MongoDB

- Insert Examples

```
> docs = [
        { name: {firstname: "Florin", lastname: "Radulescu"}, age: 58, courses: [ {name: "BD1", year: 3}, {name: "BD2", year: 4}, , {name: "UBD", year: 4}, {name: "DMDW", year: 5}], offices: ["Vice Dean Office", "EF309"]},
        { name: {firstname: "Sorin", lastname: "Ciolofan"}, age: 37, courses: [{name: "PAJ", year: 5}, {name: "MN", year: 1}]},
        {name: {firstname: "Dan", lastname: "Schrager"}, courses: [ {name: "PC", year: 1}, {name: "ABD", year: 5}], offices: ["PR304"]}
]

> db.teachers.insertMany(docs, {ordered: true})
```

CRUD Operations in MongoDB

- Query Documents:
  - db.collection.find(query, projection)
  - db.collection.findOne(query, projection)

CRUD Operations in MongoDB

- Find all:

```
> db.teachers.find()
```

- Find one:

```
> db.teachers.findOne()
```

- Find all under 40 years:

```
> db.teachers.find({age: {$lte: 40}}, {name: 1})
```

- Find all over 35:

```
> db.teachers.find({age: {$gte: 35}}, {name: 1})
```

- Find all with age between 40 and 60 years:

```
> db.teachers.find({age: {$gte: 40, $lte: 60}}, {name: 1})
```


CRUD Operations in MongoDB

- Find all who have the office EF309:

```
> db.teachers.find({offices: "EF309"}, {name: 1})
```

- Find all who do not have an office:

```
> db.teachers.find({offices: {$exists: false}}, {name: 1})
```

- Find all who teach BD1:

```
> db.teachers.find({"courses.name": "BD1"}, {name: 1})
```

- Find all teachers whose courses starting with DM (regex):

```
> db.teachers.find({"courses.name": /^DM/}, {name: 1, "courses.name": 1})
```

- Find all who have courses at the 1 year or his age is over 50:

```
> db.teachers.find({$or: [{"courses.year": 1}, {age: {$gt: 50}}]}, {name: 1})
```


CRUD Operations in MongoDB

- Find all who have courses at the 5 year and his age is under 50:

```
> db.teachers.find({$and: [{"courses.year": 5}, {age: {$lt: 50}}]}, {name: 1})
```

- Find all that have 2 offices:

```
db.teachers.find({offices: {$size: 2}}, {name: 1})
```

- Find all that has as his first office EF309:

```
db.teachers.find({"offices.0": "EF309"}, {name: 1})
```

- Find all that have offices in both EF and PR:

```
db.teachers.find({offices: {$all: [/^EF/, /^PR/]}}, {name: 1})
```


CRUD Operations in MongoDB

- Update:
  - db.collection.update(<filter>, <update>, <options>)
  - db.collection.updateOne(<filter>, <update>, <options>)
  - db.collection.updateMany(<filter>, <update>, <options>)
  - db.collection.replaceOne(<filter>, <replacement>, <options>)

CRUD Operations in MongoDB

- Update courses for teachers who are 32 years old:

```
> db.teachers.updateOne({age: 32}, {$set: {courses: {name: 'UBD', year: 4}}})
```

- Update multiple:

```
> db.teachers.update({age: 32}, {$set: {courses: {name: 'UBD', year: 4}}},
{multi: true})
```

- Update courses for all the teachers over 50 years:

```
> db.teachers.updateMany({age: {$gte: 50}}, {$set: {courses: {name: 'UBD',
year: 4}}})
```

```
> db.teachers.update({age: {$gte: 50}}, {$set: {courses: {name: 'UBD', year:
4}}}, {multi: true})
```


CRUD Operations in MongoDB

- Update a nested document, change the name of course BD1 to UBD for Ciprian

```
> db.teachers.update({"name.firstname": "Ciprian", "courses.name": "BD1"},
{$set: {"courses.$.name": 'UBD'}})
```

```
> db.teachers.update({"name.firstname": "Ciprian", "courses.name": "UBD"},
{$set: {"courses.$.name": 'BD1'}})
```

```
> db.teachers.find({"name.firstname": "Ciprian"}, {_id: 0, "courses.name": 1})
```


CRUD Operations in MongoDB

- Add a new course for Dan

```
> db.teachers.update({"name.firstname": "Dan"}, {$push: {courses: {name:
'BD1', year: 3}}})
```

```
> db.teachers.find({"name.firstname": "Dan"}, { courses: 1})
```

- Add middle name to Ciprian

```
> db.teachers.update({"name.firstname": "Ciprian"}, {$set:
{"name.middlename": 'Octavian'}})
```

```
> db.teachers.find({"name.firstname": "Ciprian"}, { name: 1})
```

- Remove middlename from Ciprian

```
> db.teachers.update({"name.firstname": "Ciprian"}, {$unset: {
"name.middlename": ""}})
```

CRUD Operations in MongoDB

- Delete:
  - db.collection.remove(<query>, <justOne>)
  - db.collection.deleteOne(<filter>)
  - db.collection.deleteMany(<filter>)
- Drop a collection
  - db.collection.drop()
- Drop a database
  - db.dropDatabase()

CRUD Operations in MongoDB

- Delete one

```
> db.teachers.remove({"name.firstname": "Ciprian"})

> db.teachers.deleteOne({"name.firstname": "Sorin"})
```

- Delete many

```
> db.teachers.deleteMany({age: {$gte: 50}})

> db.teachers.remove({age: {$gte: 50}}, {justOne: false})
```

- Remove everything

```
> db.teachers.remove({})

> db.teachers.deleteMany({})
```


CRUD Operations in MongoDB

- Drop collection

```
> db.teachers.drop()
```

- Drop database

```
> db.dropDatabase()
```


Indexes in MongoDB

- The _id field acts as a primary key and, when build, an index is automatically created
- Secondary indexes can be added on any field
- Secondary indexes can be:
  - Simple indexes on one field
  - Compound indexes on multiple fields
  - Multikey indexes for array
  - Geospatial indexes for geospatial coordinates
  - Text indexes
  - Hashed indexes to support hash based sharding

Indexes in MongoDB

- Simple indexes on one field:

```
> db.collection.createIndex({"label_1": 1})
```

- Compound indexes on multiple fields

```
> db.collection.createIndex({"label_1": 1, "label_2": -1, ...})
```

- Multikey indexes for array

```
> db.collection.createIndex({"label_1.nested_label_1": 1})
```


Indexes in MongoDB

- Geospatial indexes for geospatial coordinates

```
> db.collection.createIndex({"label": "2d"})
> db.collection.createIndex({"label": "2dsphere"})
```

- Text indexes (one per collection, but can be compound)

```
> db.collection.createIndex({"flabel": "text"})

> db.collection.createIndex({"label_1": "text", "label_2": "text",...})
```

- Hashed indexes to support hash based sharding

```
db.collection.createIndex({"label_1": "hashed"})
```


Indexes in MongoDB

- Simple index

```
> db.documents.createIndex({author: 1})
```

- Compound index

```
> db.documents.createIndex({gender: 1, age: 1})
```

- Multikey index

```
> db.documents.createIndex({"words.word": 1})
```

- Geospatial index

```
> db.documents.createIndex({"geoLocation": "2d"})
```


Indexes in MongoDB

- Text index

```
> db.documents.createIndex({"lemmaText": "text"})
```

- Hashed index

```
> db.documents.createIndex({"date": "hashed"})
```

- Get all indexes for a collection

```
> db.documents.getIndexes()
```

- Drop an Index

```
> db.documents.dropIndex("lemmaText_text")
```

- Drop all indexes except for primary key

```
> db.documents.dropIndexes()
```

Indexes in MongoDB

- Searching with Geospatial indexes
  - To find all the documents near a point (±1)

```
> db.documents.find( { geoLocation: { $near: [25, 25], $minDistance: 0, $maxDistance: 5 } }, {geoLocation: 1} )
```

  - To find all the documents in the circle with the center [25, 10], and radius 3

```
> db.documents.find( { geoLocation: { $within: { $center: [[25, 10], 3] } } }, {geoLocation: 1} )
```

  - To find all the documents in the rectangle defined by [25, 40], [30, 45]

```
> db.documents.find( { geoLocation: { $within: { $box: [[25, 40], [30, 45]] } } }, { geoLocation: 1 } )
```

  - To find all the documents in the polygon defined by a polygon with 3 points

```
> db.documents.find( { geoLocation: { $within: { $polygon: [[20, 40], [40, 50], [30, 30]] } } }, { geoLocation: 1 } )
```


Indexes in MongoDB

- Searching with Text indexes

```
{
 $text:
  {
    $search: <string>,
    $language: <string>,
    $caseSensitive: <boolean>,
    $diacriticSensitive: <boolean>
  }
}
```


Indexes in MongoDB

- Searching with Text indexes
- Search for a word

```
> db.documents.find( { $text: { $search: "cup", $language: "english", $caseSensitive: false, $diacriticSensitive: false } } , { lemmaText: 1 } )
```

- Search for a multiple word (with an or between them)

```
> db.documents.find( { $text: { $search: "cup winner", $language: "english", $caseSensitive: false, $diacriticSensitive: false } } , { lemmaText: 1 } )
```


Indexes in MongoDB

- Searching with Text indexes
- Search for a Phrase

```
db.documents.find( { $text: { $search: "\"cup winner\"", $language: "english", $caseSensitive: false, $diacriticSensitive: false } } , { lemmaText: 1 } )
```

- Search with Text Search Score

```
db.documents.find( { $text: { $search: "cup winner", $language: "english", $caseSensitive: false, $diacriticSensitive: false } } , { score: { $meta: "textScore" }, lemmaText: 1, _id: 0})
```


Aggregation in MongoDB

- Aggregation in MongoDB can be done using:
  - Single Purpose Aggregation methods
  - Aggregation Pipeline
  - Map-Reduce

Single purpose aggregation methods

- Count: db.collection.count(query)

```
> db.documents.countDocuments({lemmaTextLength: 10})
```

- Distinct: db.collection.distinct(field, query)

```
> db.documents.distinct("age")

> db.documents.distinct("age", {gender: "female"}).sort()

> db.documents.distinct("age", {gender: "female"}).reverse()
```


Aggregation Pipeline (I)

- MongoDB's aggregation framework is modeled on the concept of data processing pipelines
- Documents enter a multi-stage pipeline that transforms the documents into an aggregated result
- The most basic pipeline stages provide filters that operate like queries and document transformations that modify the form of the output document
- The operation's order matters

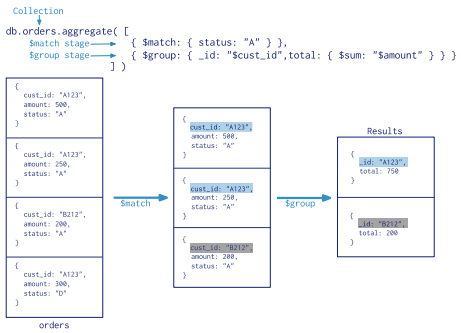
Aggregation Pipeline (I)

- Other pipeline operations provide tools for grouping and sorting documents by specific field or fields as well as tools for aggregating the contents of arrays, including arrays of documents
- In addition, pipeline stages can use operators for tasks such as calculating the average or concatenating a string
- The pipeline provides efficient data aggregation using native operations within MongoDB
- It is the preferred method for data aggregation in MongoDB
- The aggregation pipeline can operate on a sharded collection

Aggregation Pipeline

- The aggregation pipeline can use indexes to improve its performance during some of its stages
- Aggregation pipeline operations have an optimization phase which attempts to reshape the pipeline for improved performance
- The aggregation pipeline can determine if it requires only a subset of the fields in the documents to obtain the results
- If so, the pipeline will only use those required fields, reducing the amount of data passing through the pipeline

Aggregation Pipeline



Aggregation Pipeline

- Aggregation Pipeline (minimal) syntax – the operations' order matters

```
db.collection.aggregate(
   { $match: <query> },
   { $unwind: <array> },
   { $project: <projection>},
   { $group: <aggregation_group, aggregation_functions> },
   { $sort: <sorting_fields> },
   { $limit: <number>}
)
```


Aggregation Pipeline

- Extract Top-K Keywords

```
TFIDF_PA_TopKWords = function(q, k){
    var docs = Array();
    noDocs = db.documents.count(q);
    // aggregate function that computes the number of times the word appears in the corpus and the TF sum
    db.documents.aggregate([
      { $match: q },
      { $unwind: "$words" },
      { $project: { word: '$words.word', wtf: '$words.tf' } },
      { $group: { _id: "$word", sum_tf: { $sum: "$wtf" }, count: { $sum: 1 } } }
    ]).forEach(function(elem){
      var tfidf = elem.sum_tf * (1 + Math.log(noDocs/elem.count));
      var doc = {word: elem._id, TFIDF: tfidf};
      docs.push(doc);
    });
    db.vocabulary.insert(docs);
    var items = db.vocabulary.find().sort({TFIDF: -1}).limit(k);
    return items;
  }
```

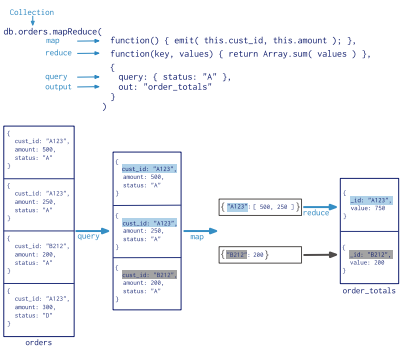

Map Reduce

- MongoDB also provides map-reduce operations to perform aggregation (deprecated)
- In general, map-reduce operations have two phases:
  - A map stage that processes each document and emits one or more objects for each input document
  - A reduce phase that combines the output of the map operation
- Optionally, map-reduce can have a finalize stage to make final modifications to the result
- Like other aggregation operations, map-reduce can specify a query condition to select the input documents as well as sort and limit the results.

Map Reduce

- Map-reduce uses custom JavaScript functions to perform the
  - Map operation
  - Reduce operations
  - As well as the optional finalize operation
- While the custom JavaScript provide great flexibility compared to the aggregation pipeline, in general, map-reduce is less efficient and more complex than the aggregation pipeline
- Map-reduce can operate on a sharded collection
- Map-reduce operations can also output to a sharded collection

Map Reduce



Map Reduce

- Map Reduce (minimal) syntax

```
db.collection.mapReduce(
   <mapFunction>,
   <reduceFunction>,
   {
      query: <filter_query>,
      out: <output_collection>,
      scope: <external variable>,
      finalize: <finalize_function,
      sort: <document>,
      limit: <number>,
   }
);
```

Map Reduce

- Extract Top-K Keywords

```
TFIDF_MR_TopKWords = function(q, k){
    // compute the number of times the word apear in the corpus and the TF sum
    mapFunction = function() {
                for (var idx=0; idx<this.words.length; idx++){
                  emit(this.words[idx].word, { count: 1, stf: this.words[idx].tf });
                } }
    reduceFunction = function(key, values) {
                    reducedVal = { count: 0, stf: 0 };
                    for (var idx = 0; idx < values.length; idx++) {
                      reducedVal.count += values[idx].count;
                      reducedVal.stf += values[idx].stf;
                    }
                    return reducedVal;};
    // The function  that computes the TFIDF
    finalizeFunction = function(key, reducedVal){
      reducedVal.tfidf = reducedVal.stf * (1 + Math.log(noDocs/reducedVal.count));
      return reducedVal; }
    var noDocs = db.documents.count(q);
    db.documents.mapReduce(mapFunction, reduceFunction, {query: q, out: "vocabulary", scope: {noDocs: noDocs}, finalize:
finalizeFunction});
    // print the values
    var items = db.vocabulary.find().sort({"value.tfidf": -1}).limit(k).addOption(DBQuery.Option.noTimeout);
    return items; }
```